High-performance, low-latency workloads on Red Hat OpenShift Virtualization at SiriusXM

Nate Mason
Director, Platform Operations,
SiriusXM Pandora

Juan Jose Floristan
OpenShift Principal Specialist SA,
Red Hat

Grant McCarthy
Senior Specialist Adoption Architect,
Red Hat

Red Hat

Agenda

- Introductions
- Why Red Hat OpenShift Virtualization?
- Best practices for high-performance and low-latency workloads
- Q & A

Introductions

Nate Mason	Director, Platform Operations, SiriusXM Pandora
Juan Jose Floristan	OpenShift Principal Specialist SA, Red Hat
Grant McCarthy	Senior Specialist Adoption Architect, Red Hat

Red Hat

Why Red Hat OpenShift Virtualization?

Red Hat

SiriusXM

Highlights:

- The goal was to consolidate multiple virtual platforms to eventually be self-service. Red Hat OpenShift was key in this modernization project.
- The team utilized the Migration Toolkit for Virtualization. Once migrated they found: “running a VM and a container, side by side, is heaven”.
- Sirius also utilizes A
  Automat

*“**Migration was the easiest part**. Once you have MTV [Migration Toolkit for Virtualization] speaking to an API, [the tool] it brings [the VMs] them in quickly with little to no down time. It’s the right tool for the job. Once we have the network setup, and everything placed where it needs to be, **the VM goes exactly where you need it to go and starts working immediately**. As far as migration goes, it’s a great tool.”*

- *Nate Mason, Associate Director, Unix Systems Group SiriusXM*

Flashback to Red Hat Summit 2024

[Source: Red Hat Summit keynote [starting at 22:57]

Red Hat

Pop Quiz:

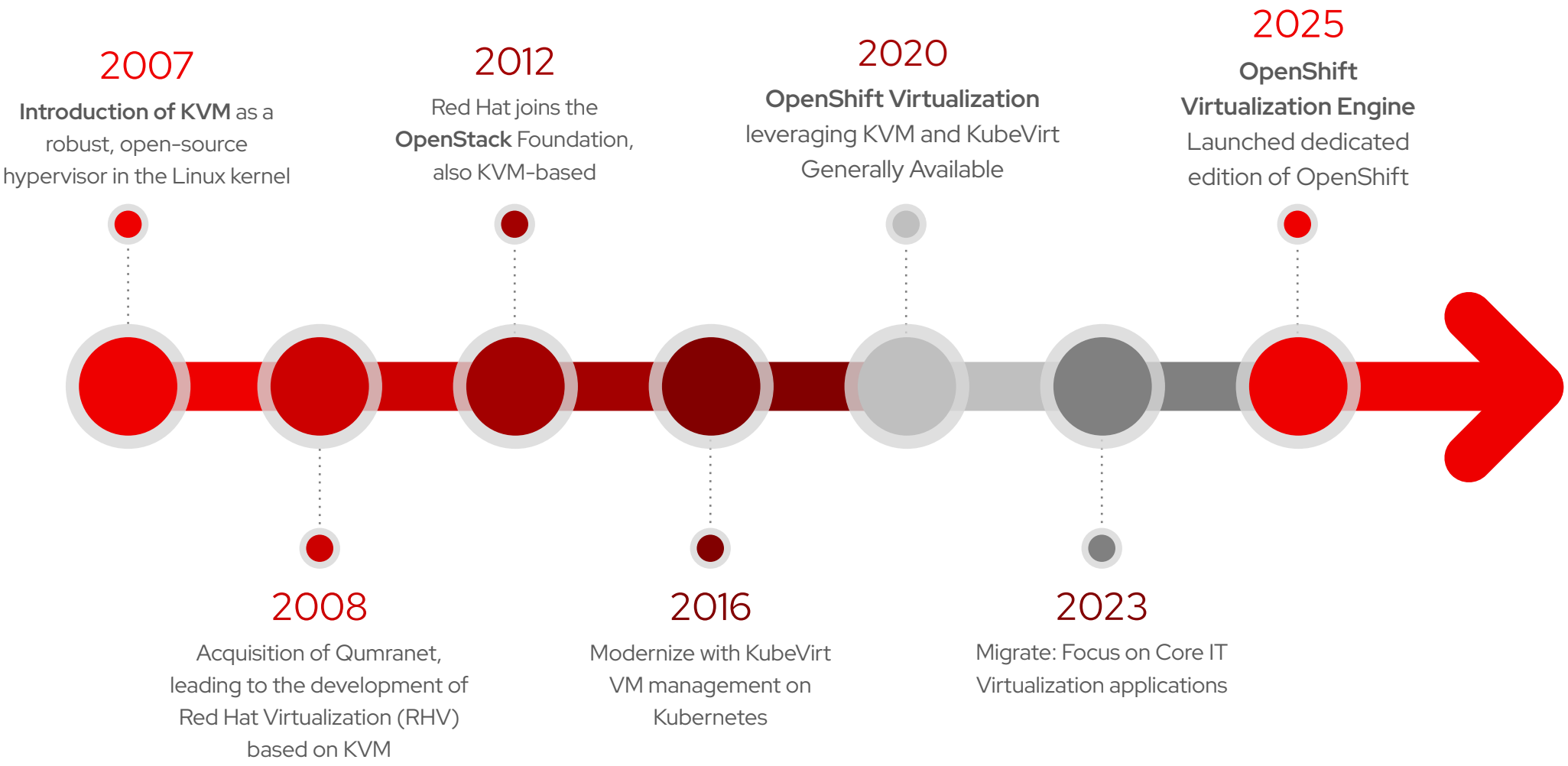
When did **Red Hat** begin working on virtualization solutions?

a) Before 2010
b) 2010 - 2015
c) 2015 - 2020
d) After 2020

Red Hat

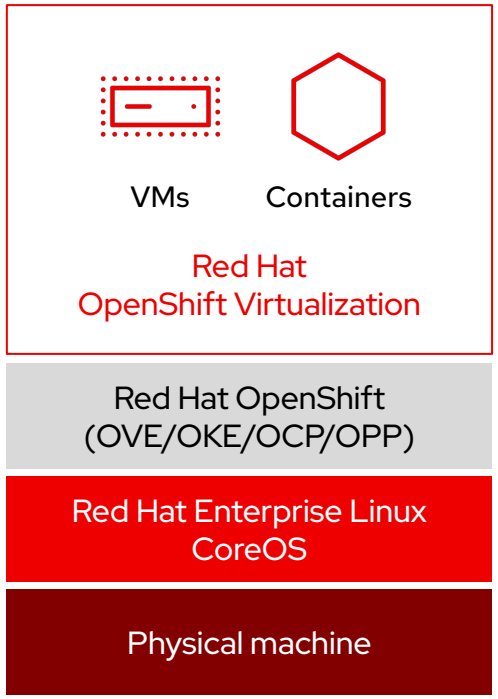
Red Hat has a long history with Virtualization

Open Source driving KVM, RHEL, OpenStack and now OpenShift Virtualization



Red Hat OpenShift Virtualization

The modern option for general purpose virtualization



- **Unified platform**
  for virtual machines and containers*
- **Consistent management**
  tools, interfaces, and APIs incl. ACM and AAP integrations
- **Performance and stability**
  of Linux, KVM, and qemu
- **Healthy open source community**
  the KubeVirt project is a top 10 CNCF active project, with 200+ contributing companies
- **Diverse ecosystem**
  of Red Hat & partner operators
- **Included feature**
  of all OpenShift subscriptions (OVE/OKE/OCP/OPP)
- **Includes Red Hat Enterprise Linux**
  guest entitlements*
- **Supports Microsoft Windows**
  guests through Microsoft SVVP
- **Inbound guest migration**
  using Ansible Automation Platform + Migration Toolkit for Virtualization, Training and Consulting
- **Virt admin focused training** DO316, EX316

*excluding OVE which is virtual machines only and includes no RHEL guest entitlements

Red Hat

Solve Near-Term Challenges, Invest in Long-Term Value

Efficiently migrate VMs now to reduce operating costs and unlock additional value in the future

MAINTAIN

vSphere Foundation

Stay as-is

- Slow evolution
- Increasing costs
- Low developer productivity

MIGRATE

OpenShift Virtualization Engine

Replace VMware

Utilize Automation and Services

- Dedicated VM management
- Reduce operating cost
- Increase IT efficiency and reliability

MODERNIZE

OpenShift

Application Modernization

Automation for Day 2 Ops

- Integrate containers and VMs
- Faster time to market
- Higher developer output

OpenShift AI

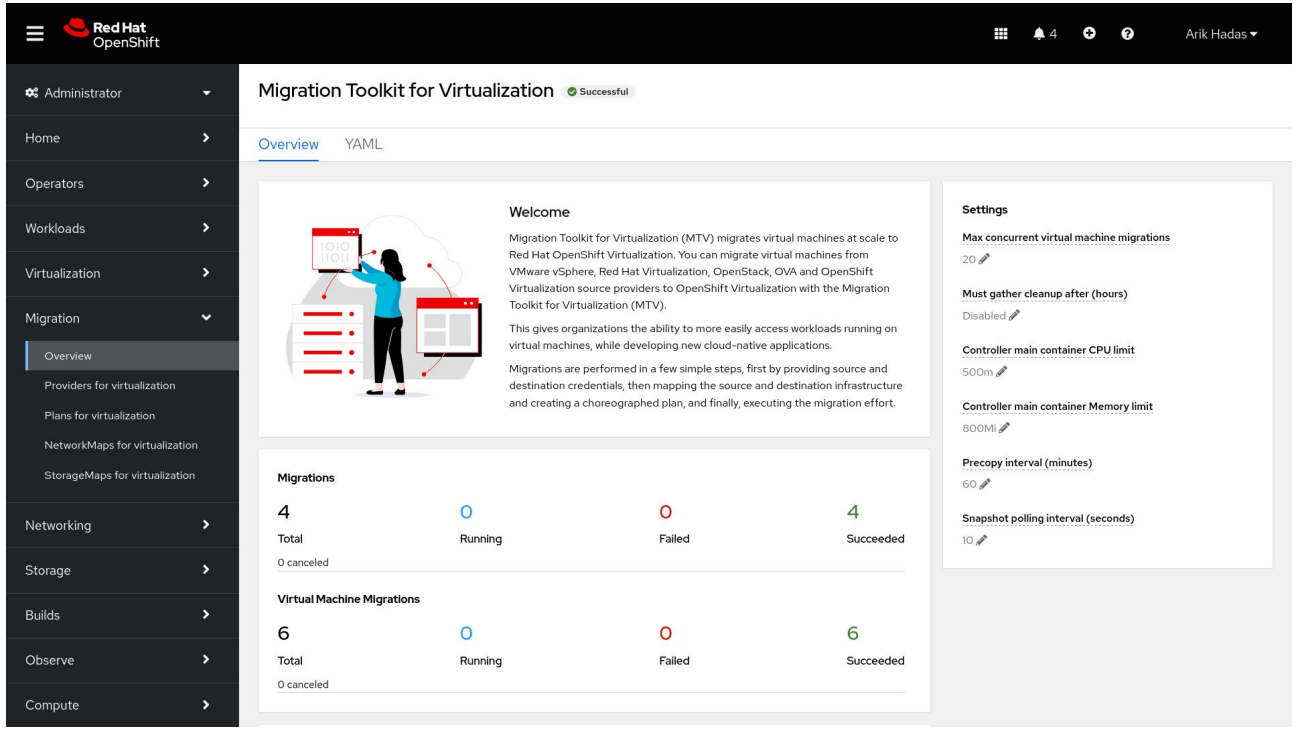
New applications

- AI/ML Management
- Increase revenue
- Reduce skill sprawl

Red Hat

Migrating VM-based applications with minimal disruption

Migration toolkit for virtualization (MTV) included with OpenShift



Easy migration of virtual machines

- Migrate virtual machines to OpenShift Virtualization in a few simple steps
- Provide source and destination credentials, map infrastructure, and create migration plans
- Currently available Sources: RHV, VMware, OpenStack, vSphere-compatible OVA, OpenShift

Red Hat

Ansible Automation Platform and OpenShift Virtualization

Better together

Ansible Automation Platform for OpenShift Virtualization enables organizations to manage and automate the lifecycle of virtual machines, orchestrating them with the broader IT infrastructure and simplify the migration process of legacy platforms.

A comprehensive platform...

Storage

- portworx by Pure Storage
- NetApp
- DELL Technologies
- Hewlett Packard Enterprise
- IBM
- lightbits
- HITACHI Inspire the Next
- INFINIDAT
- Red Hat OpenShift Data Foundation

Backup & Disaster Recovery

- COHESITY
- DELL Technologies
- veeam Kasten
- rubrik
- TRILIOVAULT
- portworx by Pure Storage
- VERITAS
- NetApp
- IBM
- ot
- storware BACKUP AND RECOVERY
- Commvault
- arctera
- Cockroach Labs

Networking & Security

- CISCO
- ISOVALENT
- TIGERA
- f5
- CHECK POINT
- paloalto NETWORKS
- flexera
- infoblox
- CROWDSTRIKE

Compute Hardware

- DELL Technologies
- Lenovo
- Hewlett Packard Enterprise
- CISCO
- IBM

Virtual Desktops

- citrix
- omnissa *In progress
- leostream

Cloud Providers

- aws
- Microsoft Azure
- ORACLE Cloud Infrastructure
- IBM
- Google Cloud

This is not an exhaustive list of ISV partners.

...with your existing technology partners

Best practices for high-performance and low-latency workloads

Red Hat

What are we trying to accomplish?

A simple but a powerful objective

“Create a deterministic, high performance and secure platform that allow us to run low latency workloads, maximize infrastructure utilization and avoid price spikes with an optimized open ecosystem”

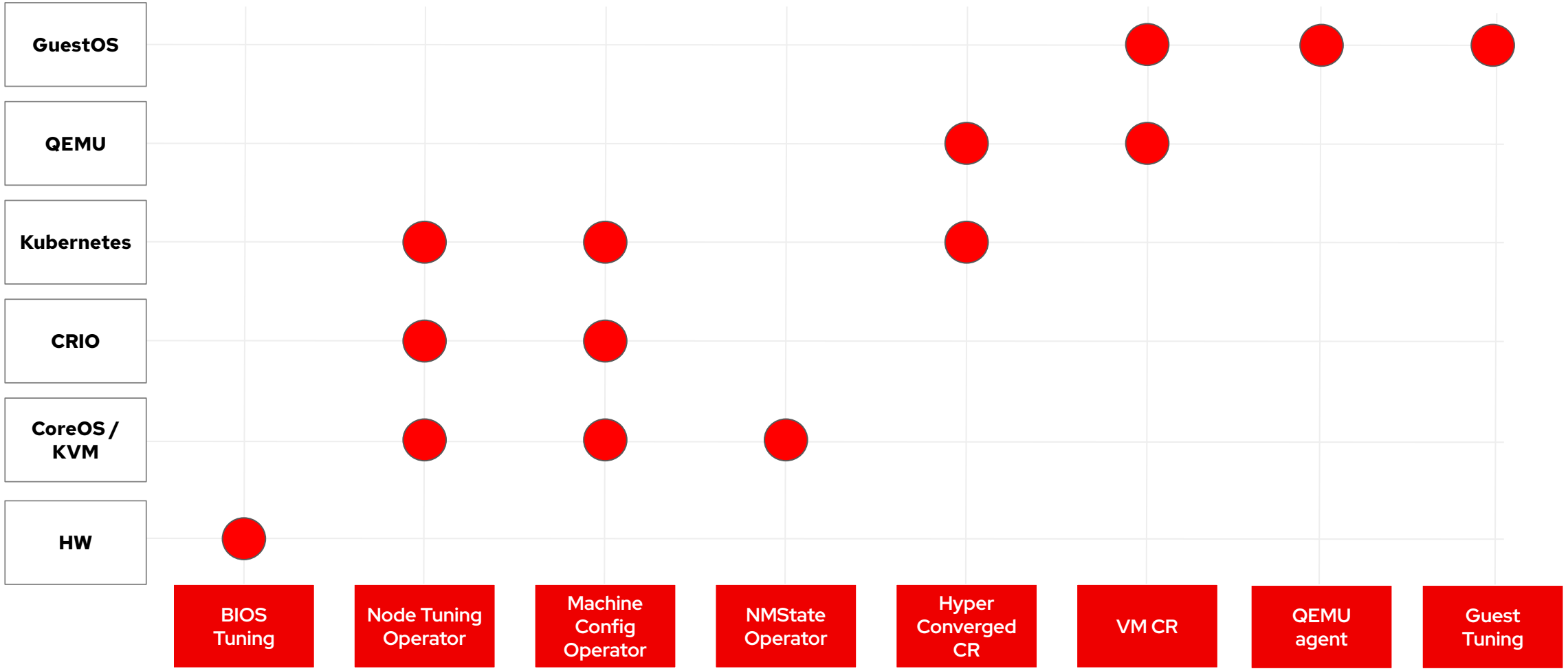
How do we start?

Designing an efficient and informed strategy

- **Understand the requirements for your use case**. Low latency means many different things in media and entertainment, trading, telco, industrial automation, healthcare...
- **Understand your limiting factors** (CPU, memory, storage I/O, network throughput...) and when they apply
- **Performance validation tools** are almost more important than the tuning itself. Have eyes everywhere, it is key to understand where and when it fails
- **Align testing and tuning with requirements and limiting factors.** Start simple and scale one step at a time, for example:
  - 1 compute node, 1 guest OS, 1 workload unit -> CPU Bound?
  - 1 compute node, 1 guest OS, N workload units -> CPU Bound?
  - 1 compute node, N guest OS, N workload units -> Network Bound?
  - N compute node, N guest OS, N workload units -> Network Bound?

What do we tune?

EVERYTHING

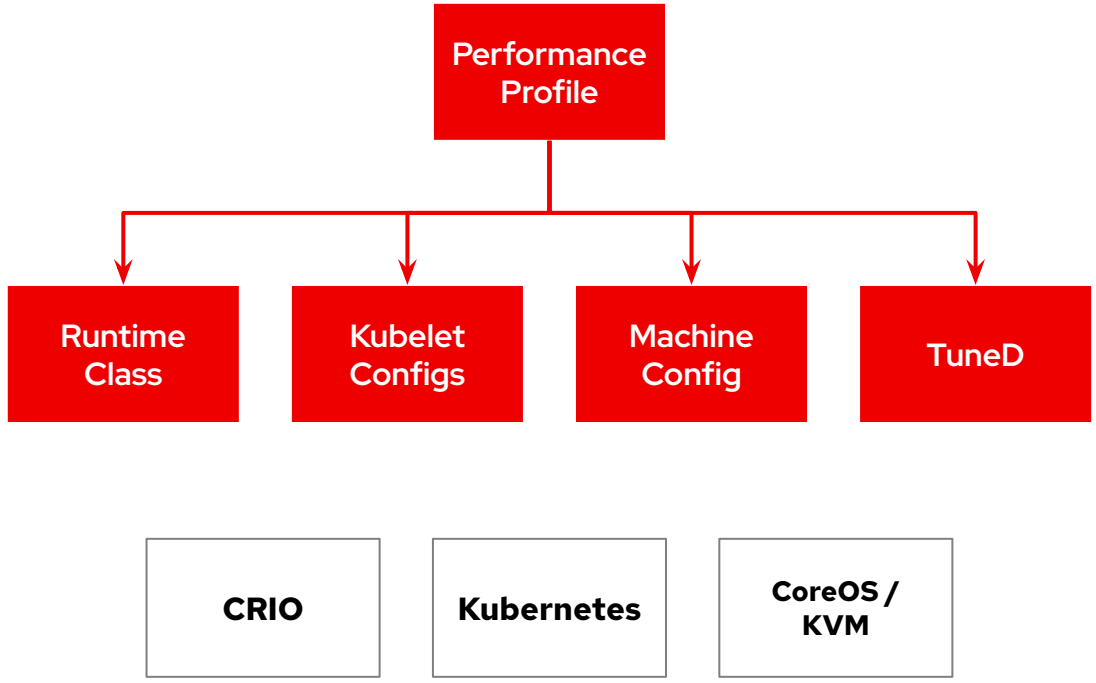


Red Hat

Node Tuning Operator

Performance Profile

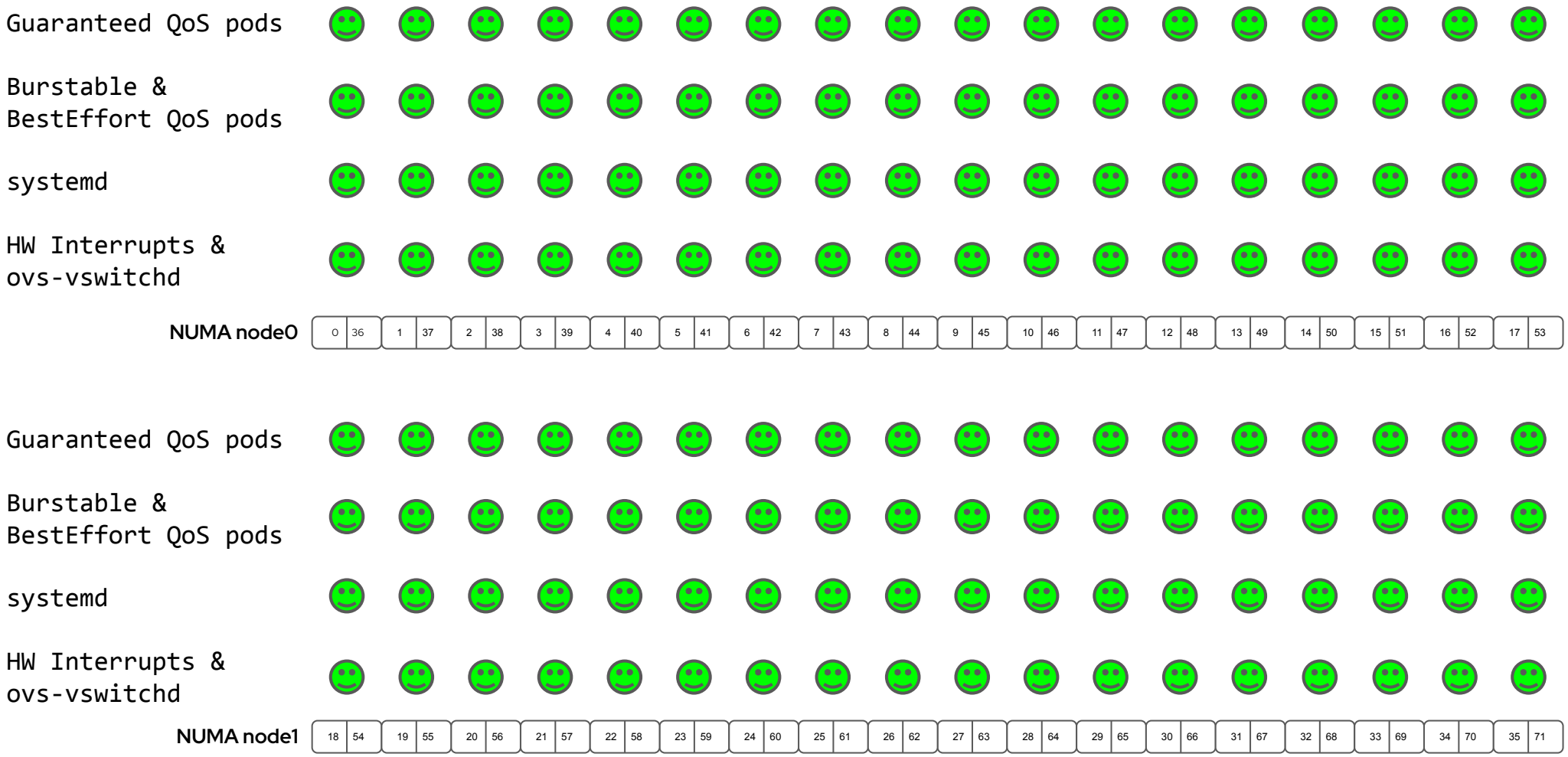
```yaml
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: <profile-name>
spec:
  globallyDisableIrqLoadBalancing: true
  cpu:
    reserved: <4 CPUs, the first Core of each NUMA node>
    isolated: <all other cpus>
    balanceIsolated: true
  additionalKernelArgs:
  - "nohz_full=<same list as isolated, above>"
  workloadHints:
    realTime: false
    highPowerConsumption: true
    perPodPowerManagement: false
  hugepages:<as required by applications, 2M or 1G>
  realTimeKernel:
    enabled: false
  numa:
    topologyPolicy: "single-numa-node"
  net:
    userLevelNetworking: false
```



Red Hat

What is allowed to run where... by default

Everything Everywhere



Red Hat

Compute Management

Performance Profile

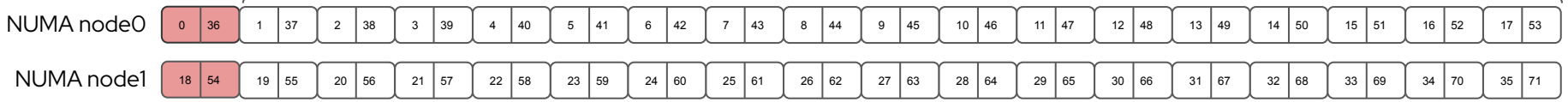
```yaml
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
spec:
  additionalKernelArgs:
    - 'kthread_cpus=1-17,19-35,37-53,55-71'
   cpu:
    # node0 CPUs: 0-17,36-53  siblings: (0,36),(1,37)...
    # node1 CPUs: 18-35,54-71
    isolated: 1-17,19-35,37-53,55-71
    reserved: 0,18,36,54
  hugepages:
    defaultHugepagesSize: 1G
    pages:
      - count: 128
        node: 0
        size: 1G
      - count: 128
        node: 1
        size: 1G
```

CPU Isolation

Pinning critical workloads to isolated cores, moving Kubelet and OS noise such as system processes and kernel threads to "reserved" house-keeping cores

Memory

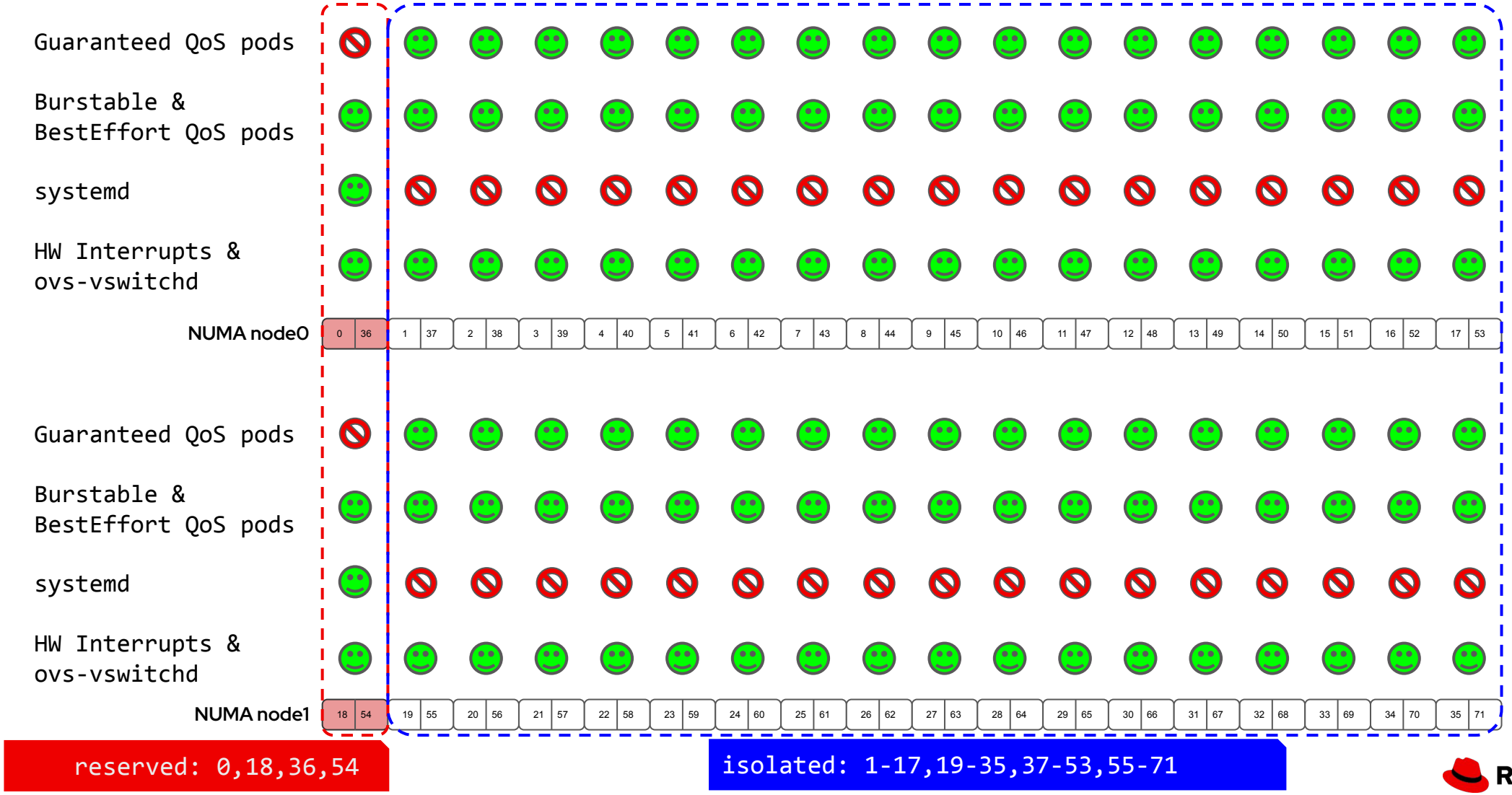
Huge pages (per NUMA node) backing up VM memory to avoid the cost of TLB misses



Red Hat

What is allowed to run where... until a Guaranteed QoS pod starts!

Reserved CPUs vs isolated CPUs



Red Hat

IRQ Management

Performance Profile

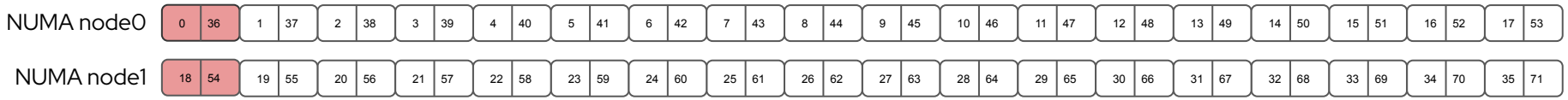
```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
spec:
  globallyDisableIrqLoadBalancing: true
  additionalKernelArgs:
    - 'nohz_full=1-17,19-35,37-53,55-71'
    - 'irqaffinity=1-17,19-35,37-53,55-71'
    #  Drivers requiring custom IRQ control
    - 'megaraid_sas.smp_affinity_enable=1'
    - 'megaraid_sas.host_tagset_enable=0'
    - 'megaraid_sas.msix_vectors=2'
    - 'lpfc.lpfc_irq_chann=2'
    - 'lpfc.lpfc_hdw_queue=2'
```

Allow applications to run at 100% capacity

Moving all IRQs to housekeeping CPUs

Additional surgery required

Some IRQ controllers lack support for an IRQ affinity setting and might always expose all online CPUs as the IRQ mask



Red Hat

What is allowed to run where... until a Guaranteed QoS pod starts!

Globally Disable IRQ Load Balancing + Work on rogue IRQs

Red Hat

Virtual Machine CR

Adding compute configurations

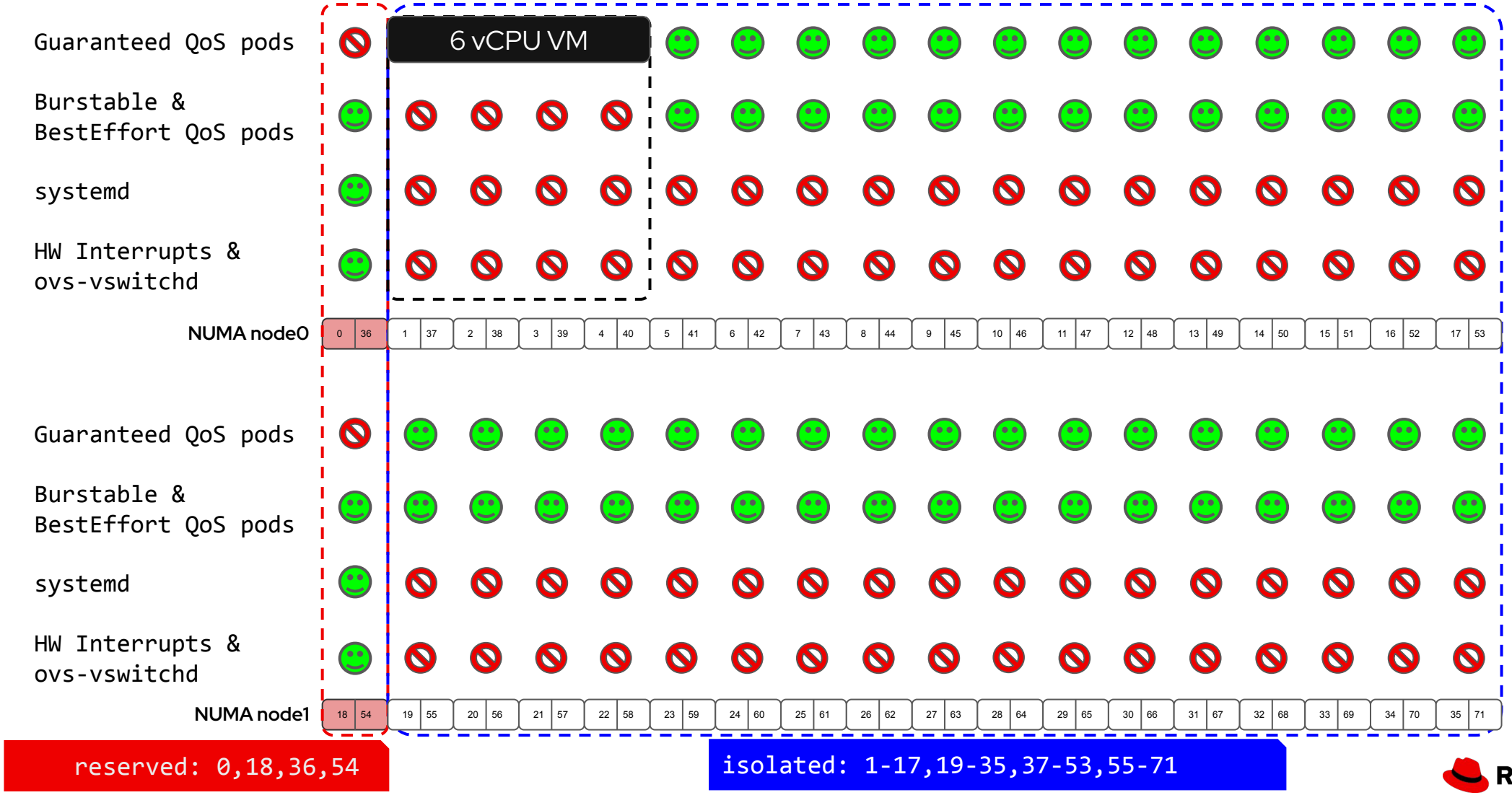
```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: low-latency-vm
spec:
  template:
    metadata:
      annotations:
        cpu-quota.crio.io: disable
    spec:
      schedulerName: topo-aware-scheduler
      domain:
        memory:
          guest: 16Gi
          hugepages:
            pageSize: 1Gi
        cpu:
          cores: 6
          dedicatedCpuPlacement: true
          isolateEmulatorThread: true
          model: host-passthrough
          numa:
            guestMappingPassthrough: {}
          sockets: 1
```

```
        ioThreadsPolicy: shared
        devices:
          autoattachMemBalloon: false
          autoattachPodInterface: false
          networkInterfaceMultiqueue: false
```


Red Hat

What is allowed to run where... when a Guaranteed QoS pod starts!

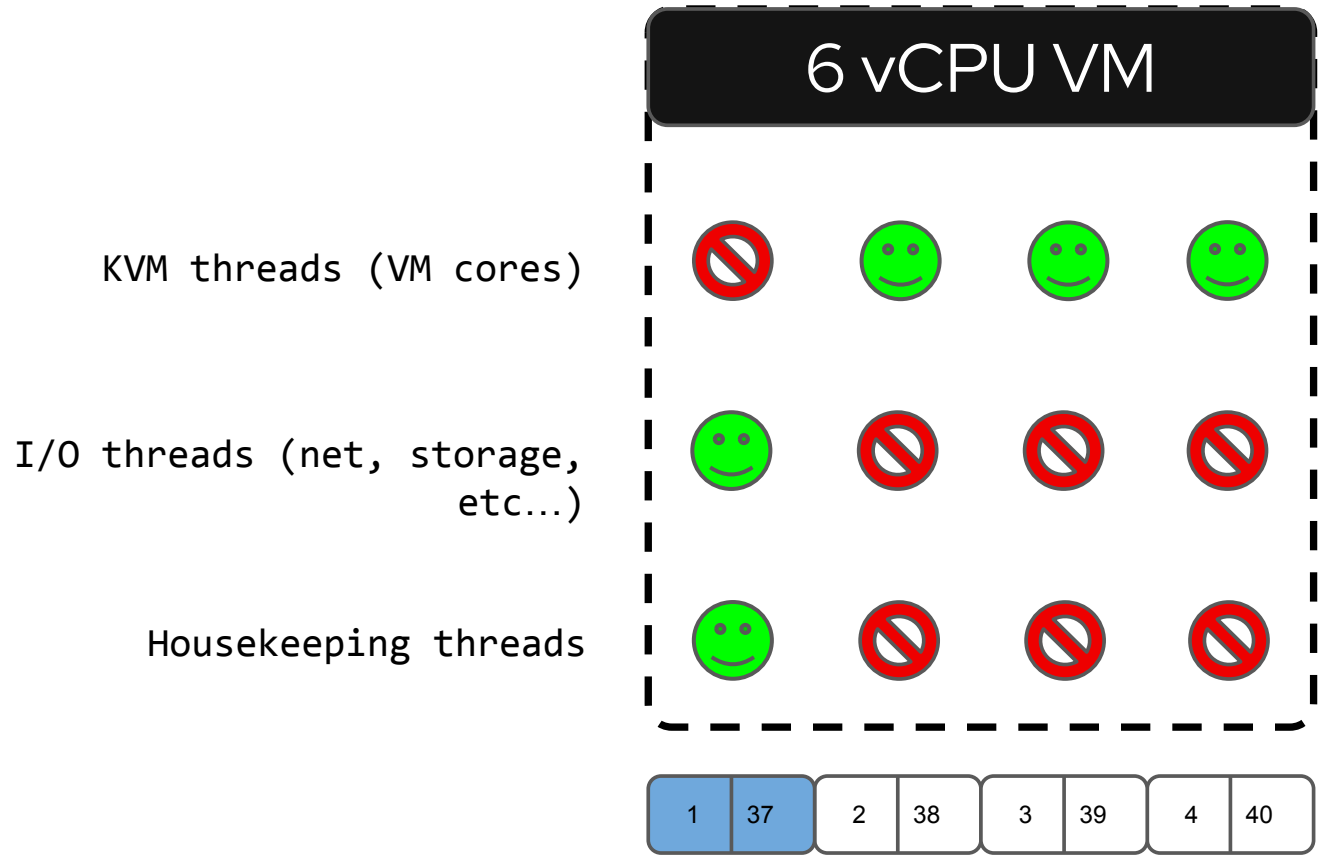
First guaranteed pod starts



Red Hat

What is allowed to run where... when a Guaranteed QoS pod starts!

First guaranteed pod starts



- Required an even number of cores
- Pairs of vCPUs to siblings in pCPUs
- Linux based GuestOS can do CPU pinning too
- Windows scheduler is a leap of faith

Red Hat

IRQ Management

Performance Profile

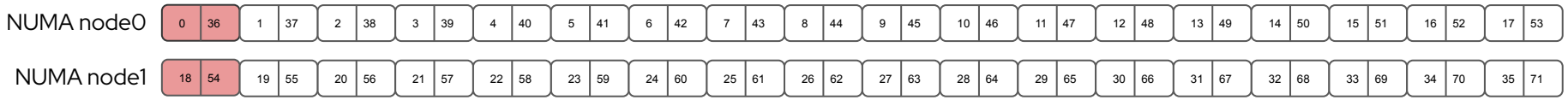
```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
spec:
  globallyDisableIrqLoadBalancing: false
  additionalKernelArgs:
    - 'nohz_full=1-17,19-35,37-53,55-71'
    #  Drivers requiring custom IRQ control
    - 'megaraid_sas.smp_affinity_enable=1'
    - 'megaraid_sas.host_tagset_enable=0'
    - 'megaraid_sas.msix_vectors=2'
    - 'lpfc.lpfc_irq_chann=2'
    - 'lpfc.lpfc_hdw_queue=2'
```

Allow applications to run at 100% capacity

Moving all IRQs out of isolated CPUs allocated to guaranteed workloads

Insufficient reserved CPUs for IRQs

For certain workloads, the reserved CPUs are not always sufficient for dealing with device interrupts, and for this reason, device interrupts are not globally disabled on the isolated CPUs



Red Hat

Virtual Machine CR

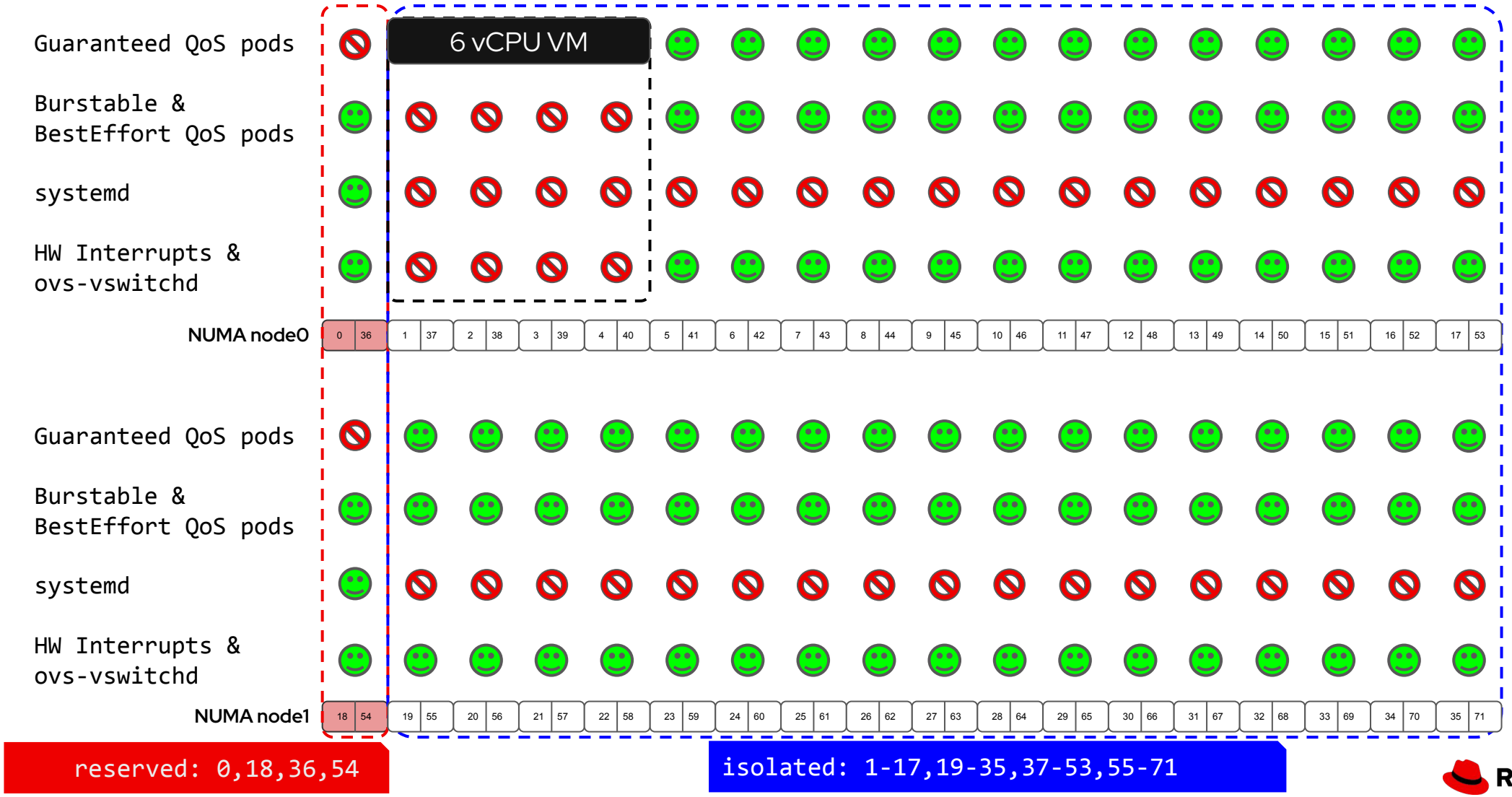
Adding IRQ management configurations

```yaml
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: low-latency-vm
spec:
  template:
    metadata:
      annotations:
        cpu-quota.crio.io: disable
        irq-load-balancing.crio.io: disable
    spec:
      schedulerName: topo-aware-scheduler
      domain:
        memory:
          guest: 16Gi
          hugepages:
            pageSize: 1Gi
        cpu:
          cores: 6
          dedicatedCpuPlacement: true
          isolateEmulatorThread: true
          model: host-passthrough
          numa:
            guestMappingPassthrough: {}
          sockets: 1
```

```yaml
      ioThreadsPolicy: shared
      devices:
        autoattachMemBalloon: false
        autoattachPodInterface: false
        networkInterfaceMultiqueue: false
```

What is allowed to run where... when a Guaranteed QoS pod starts!

First guaranteed pod starts WITHOUT Disable GIRQ LB but WITH irq-load-balancing.crio.io: "disable"



Red Hat

Power Management

Performance Profile

```yaml
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
spec:
  additionalKernelArgs:
  -  idle=poll
  workloadHints:
    realTime: false
    highPowerConsumption: true
    perPodPowerManagement: false
```

Allow applications to run at 100% power

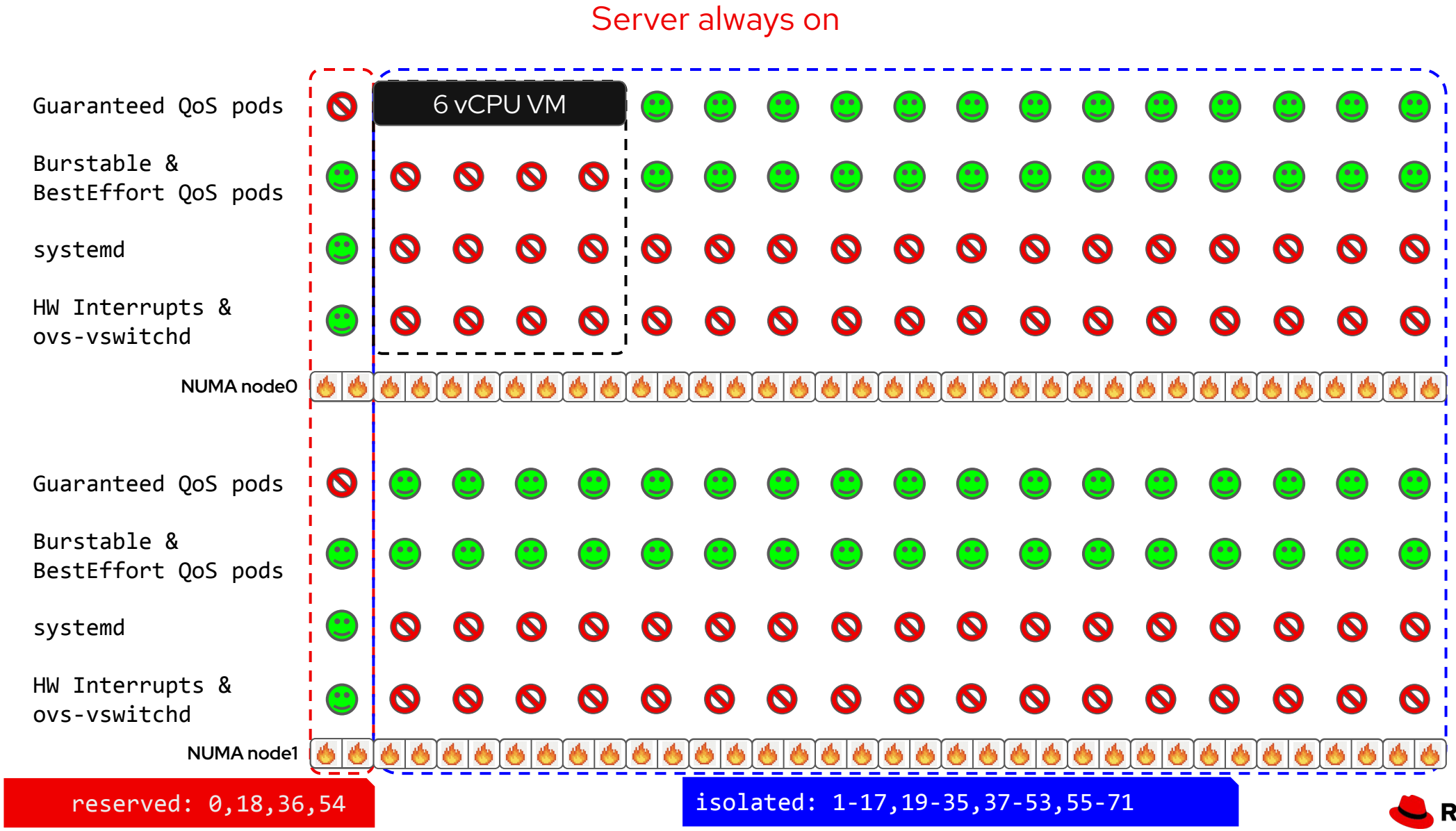
Set all CPU configurations to max performance and frequency

Keeping all CPUs always awake

Going to sleep saves power, but waking up takes time

Red Hat

What is allowed to run where... when a Guaranteed QoS pod starts!



Red Hat

Power Management

Performance Profile

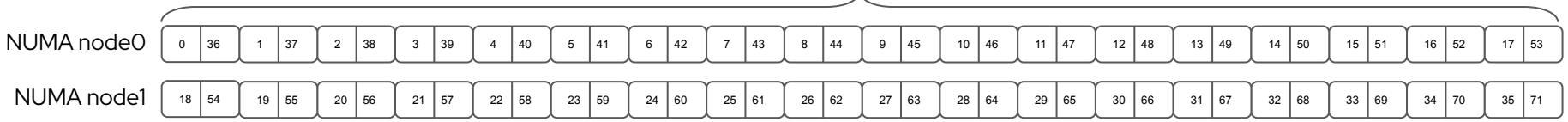
```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
spec:
  additionalKernelArgs:
  -  idle=poll
  workloadHints:
    realTime: false
    highPowerConsumption: false
    perPodPowerManagement: true
```

Allow applications to run at 100% power

Set isolated CPUs allocated to guaranteed workloads to max performance and frequency

Keeping some CPUs always awake

Going to sleep saves power, but waking up takes time



Red Hat

Virtual Machine CR

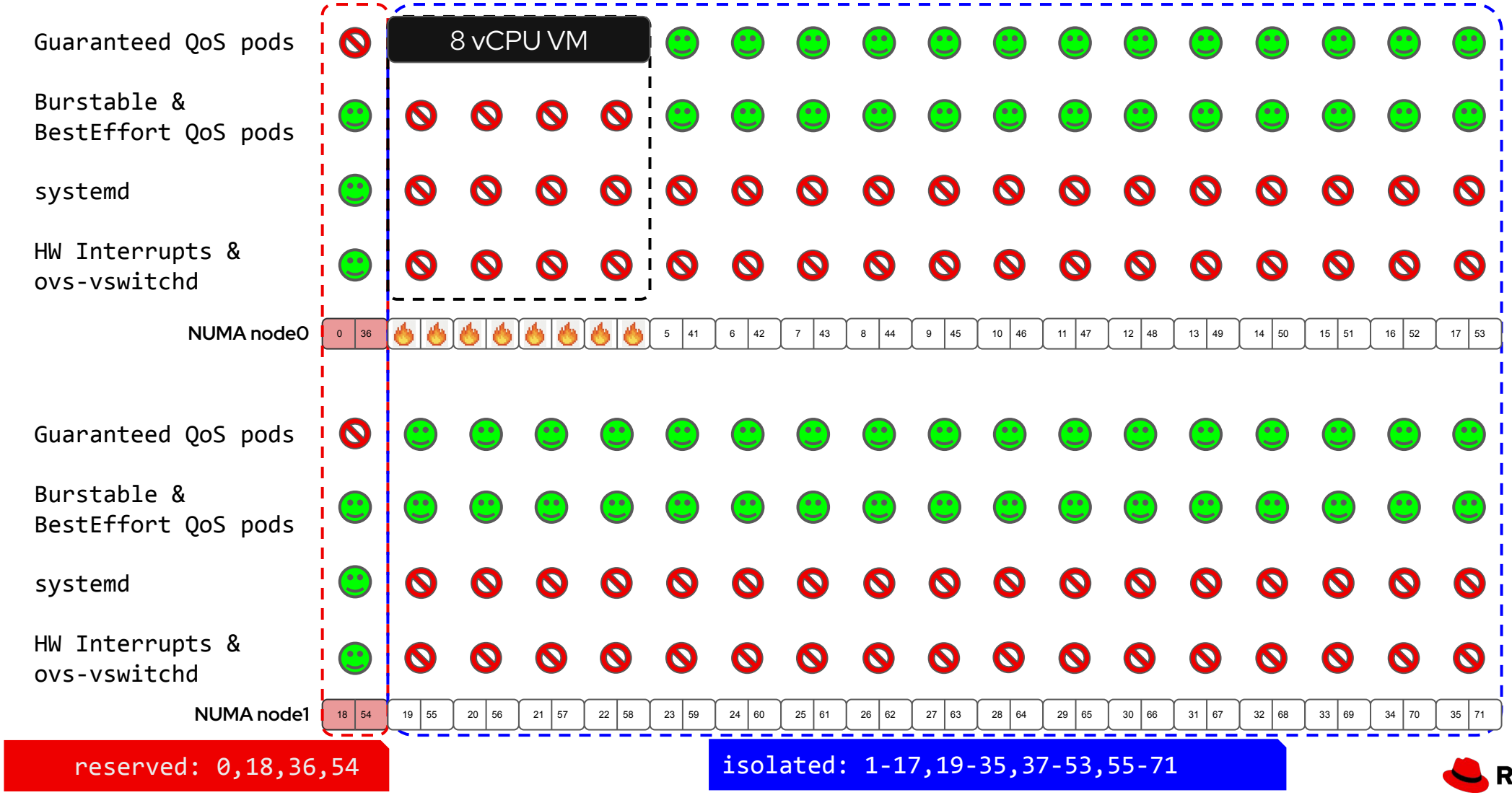
Adding “power” management configurations

```yaml
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: low-latency-vm
spec:
  template:
    metadata:
      annotations:
        cpu-quota.crio.io: disable
        irq-load-balancing.crio.io: disable
        cpu-c-states.crio.io: disable
        cpu-freq-governor.crio.io: "performance"
    spec:
      schedulerName: topo-aware-scheduler
      domain:
        memory:
          guest: 16Gi
          hugepages:
            pageSize: 1Gi
```

```yaml
        cpu:
          cores: 6
          dedicatedCpuPlacement: true
          isolateEmulatorThread: true
          model: host-passthrough
          numa:
            guestMappingPassthrough: {}
          sockets: 1
        ioThreadsPolicy: shared
        devices:
          autoattachMemBalloon: false
          autoattachPodInterface: false
          networkInterfaceMultiqueue: false
```

What is allowed to run where... when a Guaranteed QoS pod starts!

Per Pod Power Management



Red Hat

Networking

Trade between performance and features

Feature / Metric	Open vSwitch (OVS)	Linux Bridge	OVS-DPDK (roadmap)	SR-IOV
Architecture Type	Advanced multi-layer software switch (SDN)	Standard in-kernel software switch	User-space software switch with kernel bypass	Hardware-level PCIe virtualization
Data Path Location	Kernel Space (Fast path) & User Space (Control)	**Kernel** Space	**User Space** entirely (DPDK poll-mode drivers)	**Direct to Hardware** (bypasses Host OS completely)
Performance & Latency	Moderate. Context switching between user/kernel adds overhead	Baseline / Moderate. **Good enough**	High throughput, **very low latency**	Maximum throughput (line rate), **ultra-low/deterministic latency**
CPU Overhead	Moderate	**Low to Moderate** (interrupt-driven)	**High**. Cores are isolated and constantly poll for packets (100% utilization)	**Near Zero**. The physical NIC handles the switching and data movement
VM Live Migration	Yes, standard cloud orchestration supports it well	**Yes**, fully supported and simple.	**Yes**, though slightly more complex to configure	**Historically Difficult**. Tying a VM directly to a physical hardware address makes live migration complex

Questions?

Red Hat

Red Hat Summit

Thank you

linkedin.com/company/red-hat

facebook.com/redhat

instagram.com/red_hat

x.com/RedHatSummit

Red Hat